SYMBIOSIS INSTITUTE OF TECHNOLOGY, PUNE

Symbiosis International (Deemed University)
(Established under section 3 of the UGC Act, 1956)
Re-accredited by NAAC with 'A' grade (3.58/4) | Awarded Category – I by UGC

Founder: Prof. Dr. S. B. Mujumdar, M. Sc., Ph. D. (Awarded Padma Bhushan and Padma Shri by President of India)

Assignment No. 10

Subject:	DevOps Lab
TCODE:	TE7950
Name of Student	**Pushkraj P Naik**
PRN No.	**23070122169**
Branch	CSE, Batch (2023-27), L3
Academic Year & Semester	2025 – 2026, Semester VII
Date of Performance	23rd July, 2026
Title of Assignment:	Docker Volumes & Data Persistence

Theory:

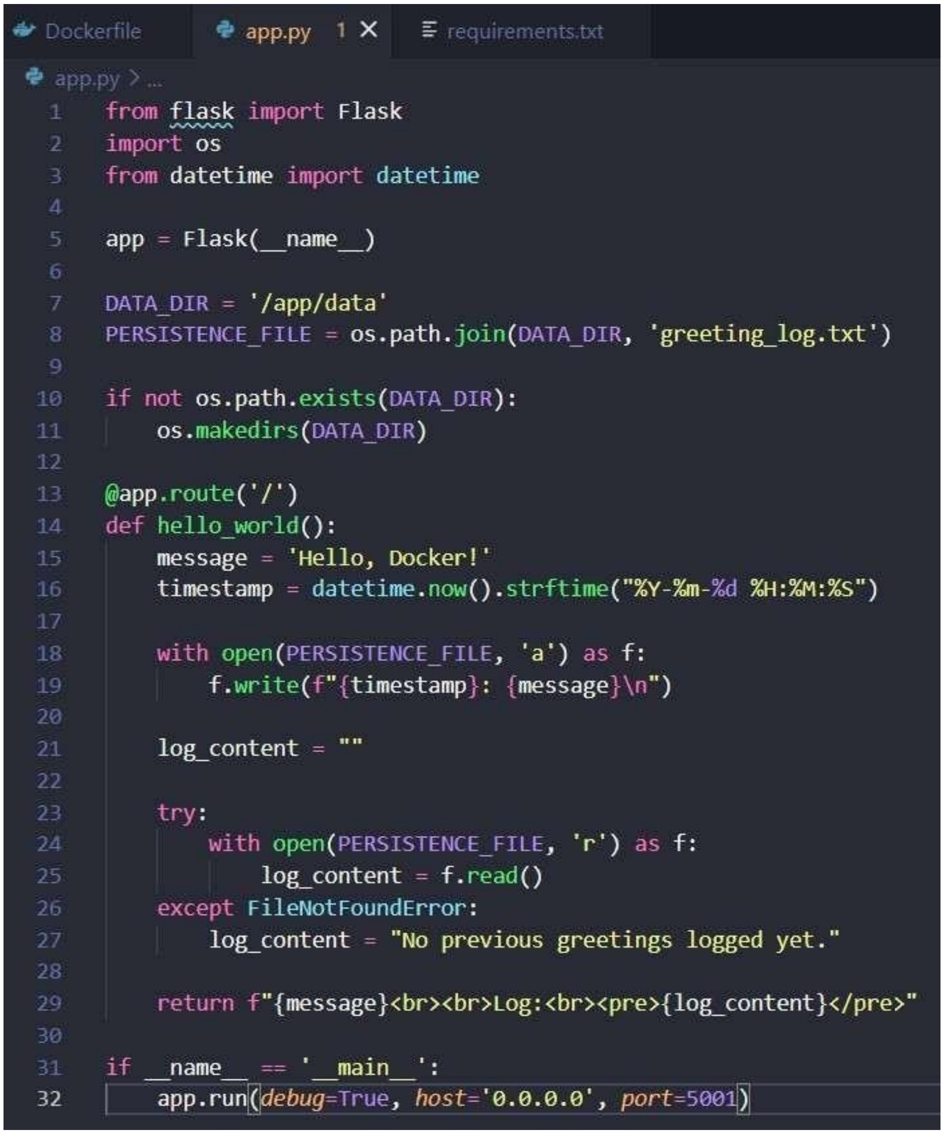
Docker volumes provide persistent storage for containers by storing data outside the container's writable layer. Unlike container storage, Docker volumes remain intact even after a container is stopped or removed, allowing data to be shared across multiple container instances. By mounting a named volume to a directory inside the container, applications can read and write data that persists independently of the container lifecycle. This makes Docker volumes ideal for storing application data, logs, databases, and other important files that should not be lost when containers are recreated.

Commands Used

- Built the updated Docker image with a new version tag: `docker build -t my-flask-app:v2 .`
- Created a Docker named volume: `docker volume create my-app-data`
- Started a container with the named volume mounted: `docker run -d -p 8082:5001 -v my-app-data:/app/data --name flaskvolume-demo my-flask-app:v2`
- Stopped the running container: `docker stop flask-volume-demo`
- Removed the stopped container: `docker rm flask-volume-demo`
- Started a new container using the same Docker volume: `docker run -d -p 8082:5001 -v my-app-data:/app/data --name flask-volume-demo2 my-flask-app:v2`

Implementation:

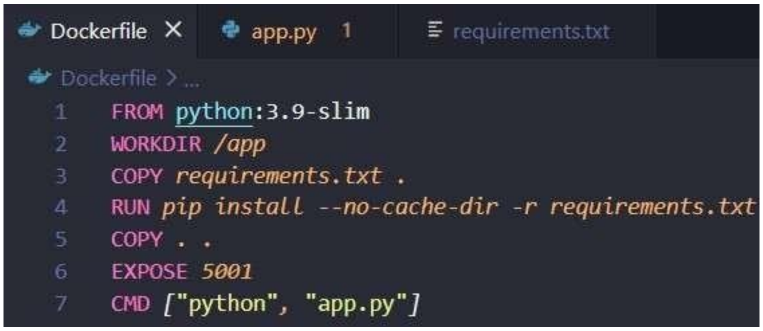
Step 1: Modify the Python Application



```python
from flask import Flask
import os
from datetime import datetime

app = Flask(__name__)

DATA_DIR = '/app/data'
PERSISTENCE_FILE = os.path.join(DATA_DIR, 'greeting_log.txt')

if not os.path.exists(DATA_DIR):
    os.makedirs(DATA_DIR)

@app.route('/')
def hello_world():
    message = 'Hello, Docker!'
    timestamp = datetime.now().strftime("%Y-%m-%d %H:%M:%S")

    with open(PERSISTENCE_FILE, 'a') as f:
        f.write(f"{timestamp}: {message}\n")

    log_content = ""

    try:
        with open(PERSISTENCE_FILE, 'r') as f:
            log_content = f.read()
    except FileNotFoundError:
        log_content = "No previous greetings logged yet."

    return f"{message}<br><br>Log:<br><pre>{log_content}</pre>"

if __name__ == '__main__':
    app.run(debug=True, host='0.0.0.0', port=5001)
```

Modified Flask Application — Updated the Flask application to write timestamped greetings to a log file inside the /app/data directory and configured the application to run on port 5001.

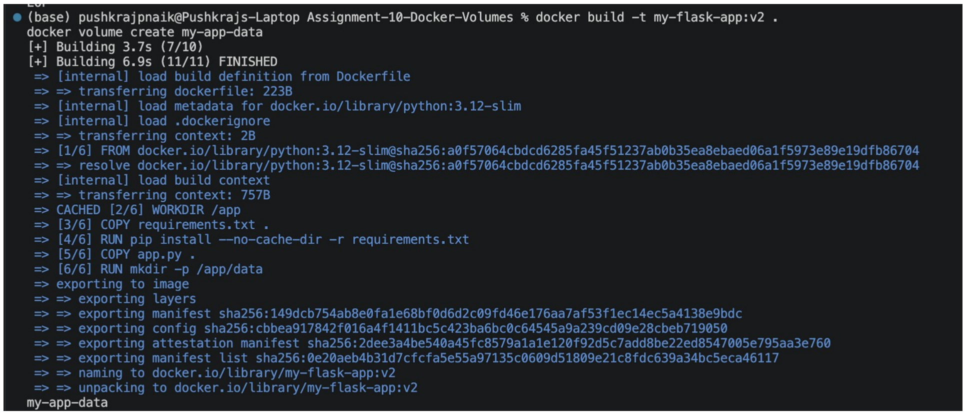
Step 2: Update the Dockerfile



```dockerfile
FROM python:3.9-slim
WORKDIR /app
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt
COPY . .
EXPOSE 5001
CMD ["python", "app.py"]
```

Updated Dockerfile — Modified the Dockerfile to expose port 5001, matching the updated Flask application configuration.

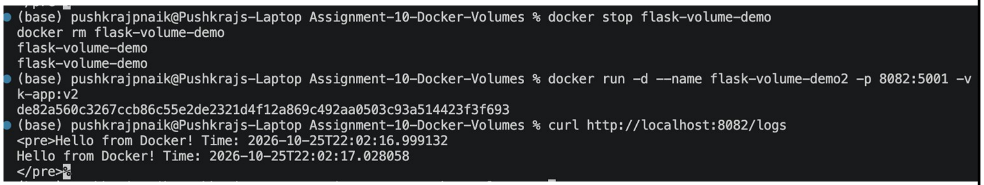
Step 3: Build the New Docker Image



```
● (base) pushkrajpnaik@Pushkrajs-Laptop Assignment-10-Docker-Volumes % docker build -t my-flask-app:v2 .
  docker volume create my-app-data
  [+] Building 3.7s (7/10)
  [+] Building 6.9s (11/11) FINISHED
   => [internal] load build definition from Dockerfile
   => => transferring dockerfile: 223B
   => [internal] load metadata for docker.io/library/python:3.12-slim
   => [internal] load .dockerignore
   => => transferring context: 2B
   => [1/6] FROM docker.io/library/python:3.12-slim@sha256:a0f57064cbdcd6285fa45f51237ab0b35ea8ebaed06a1f5973e89e19dfb86704
   => => resolve docker.io/library/python:3.12-slim@sha256:a0f57064cbdcd6285fa45f51237ab0b35ea8ebaed06a1f5973e89e19dfb86704
   => [internal] load build context
   => => transferring context: 757B
   => CACHED [2/6] WORKDIR /app
   => [3/6] COPY requirements.txt .
   => [4/6] RUN pip install --no-cache-dir -r requirements.txt
   => [5/6] COPY app.py .
   => [6/6] RUN mkdir -p /app/data
   => exporting to image
   => => exporting layers
   => => exporting manifest sha256:149dcb754ab8e0fa1e68bf0d6d2c09fd46e176aa7af53f1ec14ec5a4138e9bdc
   => => exporting config sha256:cbbea917842f016a4f1411bc5c423ba6bc0c64545a9a239cd09e28cbeb719050
   => => exporting attestation manifest sha256:2dee3a4be540a45fc8579a1a1e120f92d5c7add8be22ed8547005e795aa3e760
   => => exporting manifest list sha256:0e20aeb4b31d7cfcfa5e55a97135c0609d51809e21c8fdc639a34bc5eca46117
   => => naming to docker.io/library/my-flask-app:v2
   => => unpacking to docker.io/library/my-flask-app:v2
  my-app-data
```

Built Updated Docker Image — Built a new Docker image named my-flaskapp:v2 containing the modified Flask application configured for Docker volumebased data persistence.

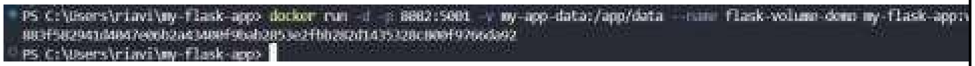
Step 4: Create a Docker Named Volume



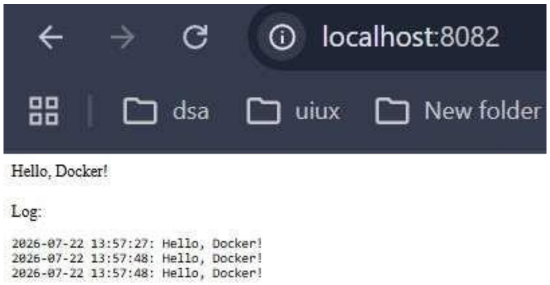
```
● (base) pushkrajpnaik@Pushkrajs-Laptop Assignment-10-Docker-Volumes % docker stop flask-volume-demo
  docker rm flask-volume-demo
  flask-volume-demo
  flask-volume-demo
● (base) pushkrajpnaik@Pushkrajs-Laptop Assignment-10-Docker-Volumes % docker run -d --name flask-volume-demo2 -p 8082:5001 -v
  k-app:v2
  de82a560c3267ccb86c55e2de2321d4f12a869c492aa0503c93a514423f3f693
● (base) pushkrajpnaik@Pushkrajs-Laptop Assignment-10-Docker-Volumes % curl http://localhost:8082/logs
  <pre>Hello from Docker! Time: 2026-10-25T22:02:16.999132
  Hello from Docker! Time: 2026-10-25T22:02:17.028058
  </pre>%
```

Created Docker Volume — Created a Docker named volume called my-app-data to store application data independently of the container, ensuring data persistence across container restarts and removal.

Step 5: Access the Application and Write Data



```
PS C:\Users\riavi\my-flask-app> docker run -d -p 8082:5001 -v my-app-data:/app/data --name flask-volume-demo my-flask-app:v
[illegible]
PS C:\Users\riavi\my-flask-app>
```



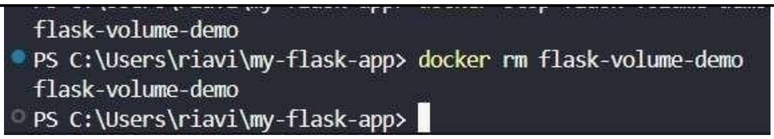
Hello, Docker!

Log:

```
2026-07-22 13:57:27: Hello, Docker!
2026-07-22 13:57:48: Hello, Docker!
2026-07-22 13:57:48: Hello, Docker!
```

Verified Data Logging — Accessed the Flask application through http://localhost:8082 and refreshed the page multiple times to verify that timestamped greetings were successfully written to the persistent log file stored in the Docker volume.

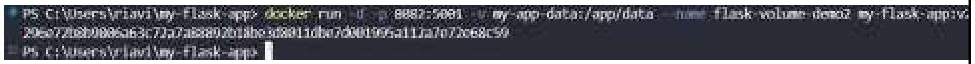
Step 6: Stop and Remove the Container



```
flask-volume-demo
PS C:\Users\riavi\my-flask-app> docker rm flask-volume-demo
flask-volume-demo
PS C:\Users\riavi\my-flask-app>
```

Stopped and Removed Container — Stopped the flask-volume-demo container and removed it using the docker stop and docker rm commands while preserving the
application data stored in the Docker named volume.

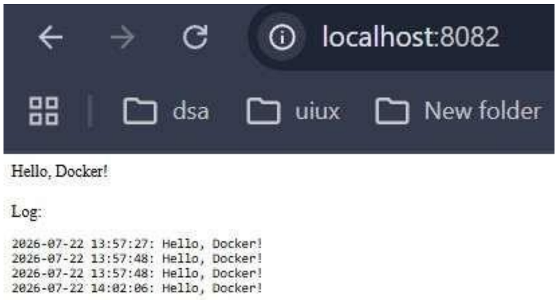
Step 7 – Run a New Container Using the Same Volume



```
PS C:\Users\riavi\my-flask-app> docker run -d -p 8082:5001 -v my-app-data:/app/data --name flask-volume-demo2 my-flask-app:v
29e072b8b9006a63c72a7a88892b18be3d8011dbe7d001995a112a7072e68c59
PS C:\Users\riavi\my-flask-app>
```

Started New Container with Existing Volume — Started a new Docker container named flask-volume-demo2 using the existing my-app-data volume to verify that previously stored application data could be reused.

Step 8: Verify Data Persistence



Hello, Docker!

Log:

```
2026-07-22 13:57:27: Hello, Docker!
2026-07-22 13:57:48: Hello, Docker!
2026-07-22 13:57:48: Hello, Docker!
2026-07-22 14:02:06: Hello, Docker!
```

Verified Data Persistence — Accessed the Flask application after starting a new container with the same Docker named volume and confirmed that the previously stored log entries were retained. This demonstrated successful data persistence across multiple container instances.

Subject Faculty Name: Prof. Arijit Dutta